5강. 쿠버네티스 개요

한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤

» Kubernetes

쿠버네티스란

쿠버네티스

컨테이너를 효율적으로 배포하고 지속해서
컨테이너 상태를 확인 및 관리하는
컨테이너 오케스트레이션 시스템

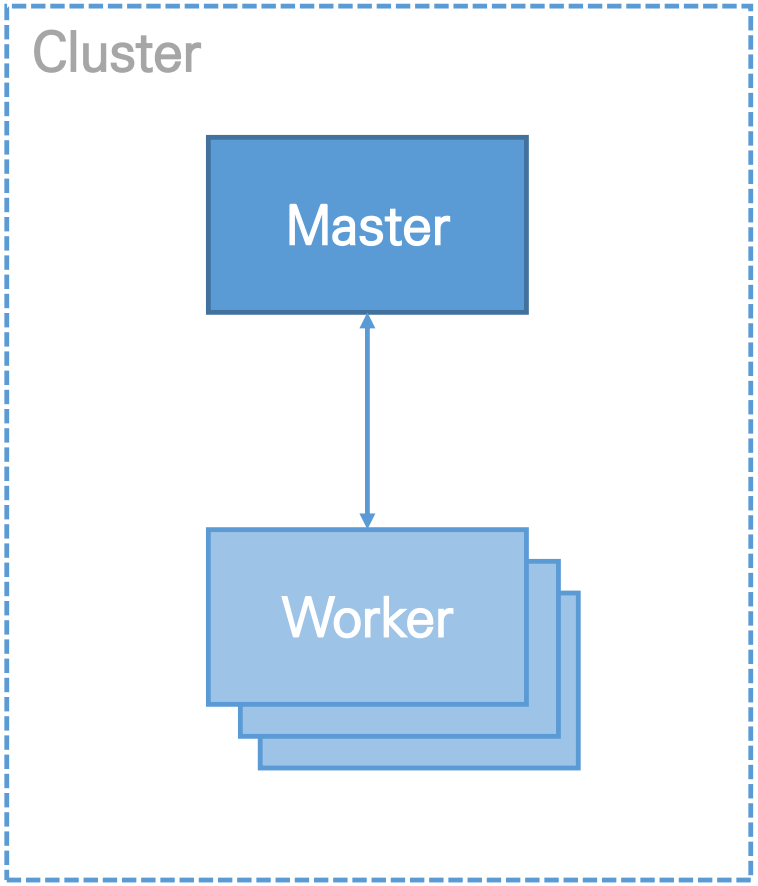
» Kubernetes

쿠버네티스 특징

- 신속한 배포
- 선언적 API
- 유연성과 확장성
- 고가용성과 성능 관리
- 어디서나 실행 가능
- 다양한 환경에서 사용 가능
- 애플리케이션 배포의 단순화
- 지속적인 상태 확인과 셀프힐링
- 오토스케일링

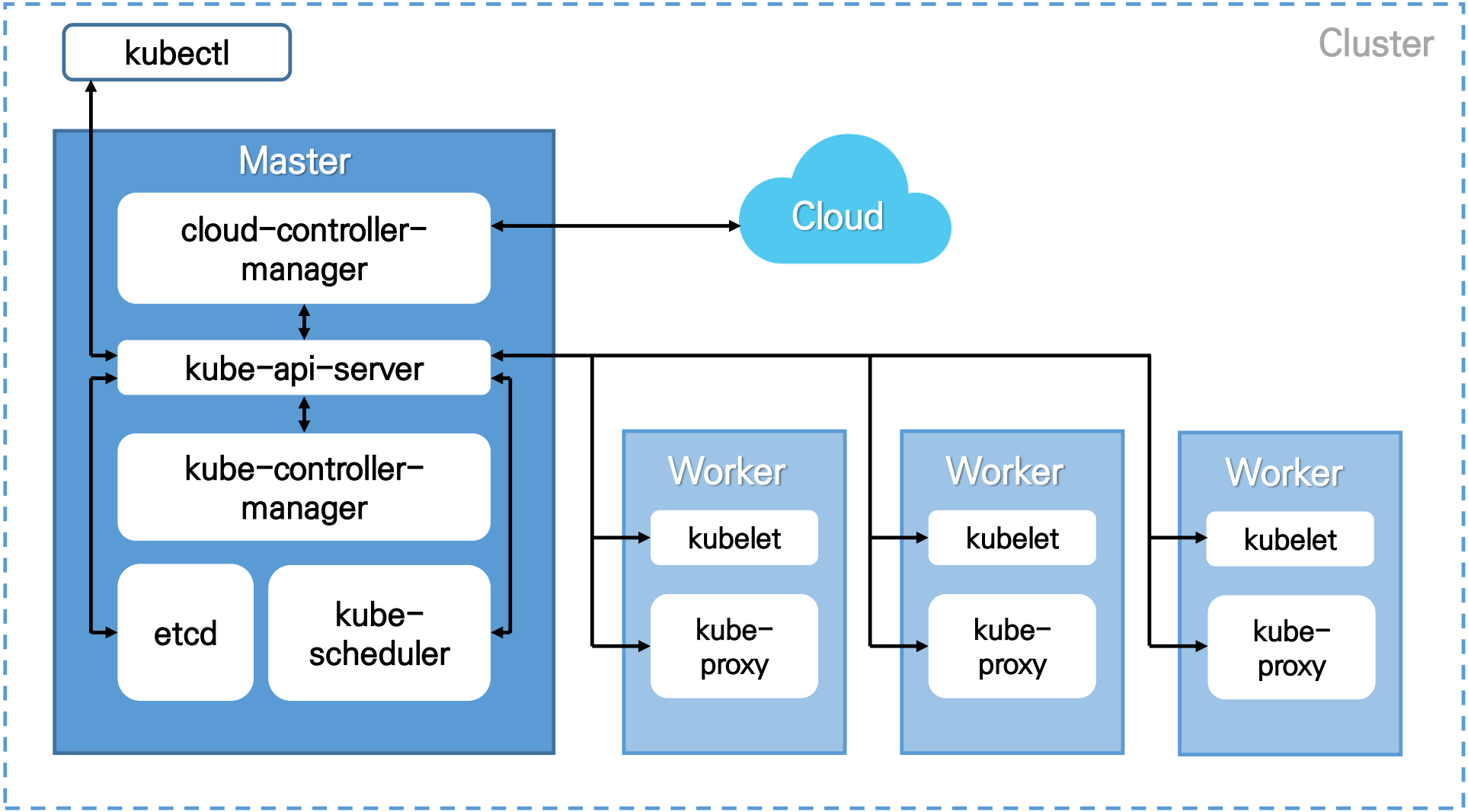
» Kubernetes Cluster 구성

클러스터 구성



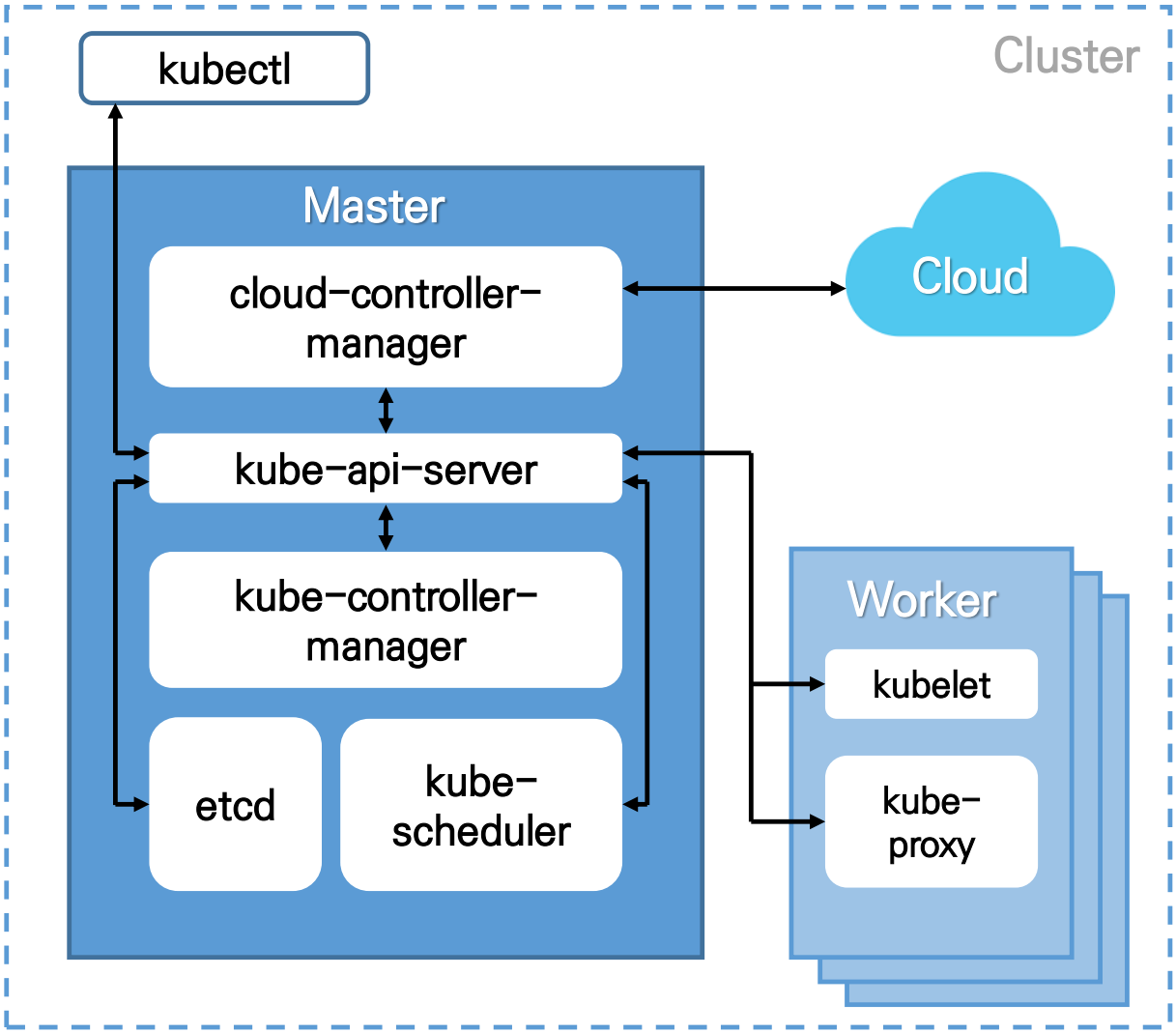
- **Cluster**
  클러스터는 컨테이너화된 애플리케이션을 실행하기 위한 일련의 노드 머신의 집합
- **Master**
  마스터 노드는 클러스터 전체를 컨트롤하며 내부에 있는 모든 노드를 관리하는 가상 머신
- **Worker**
  마스터에 의해 명령을 받고 실제 컨테이너들이 생성되고 일을 하는 가상 머신

» Kubernetes Cluster 구성



Kubernetes Cluster 구성 소개

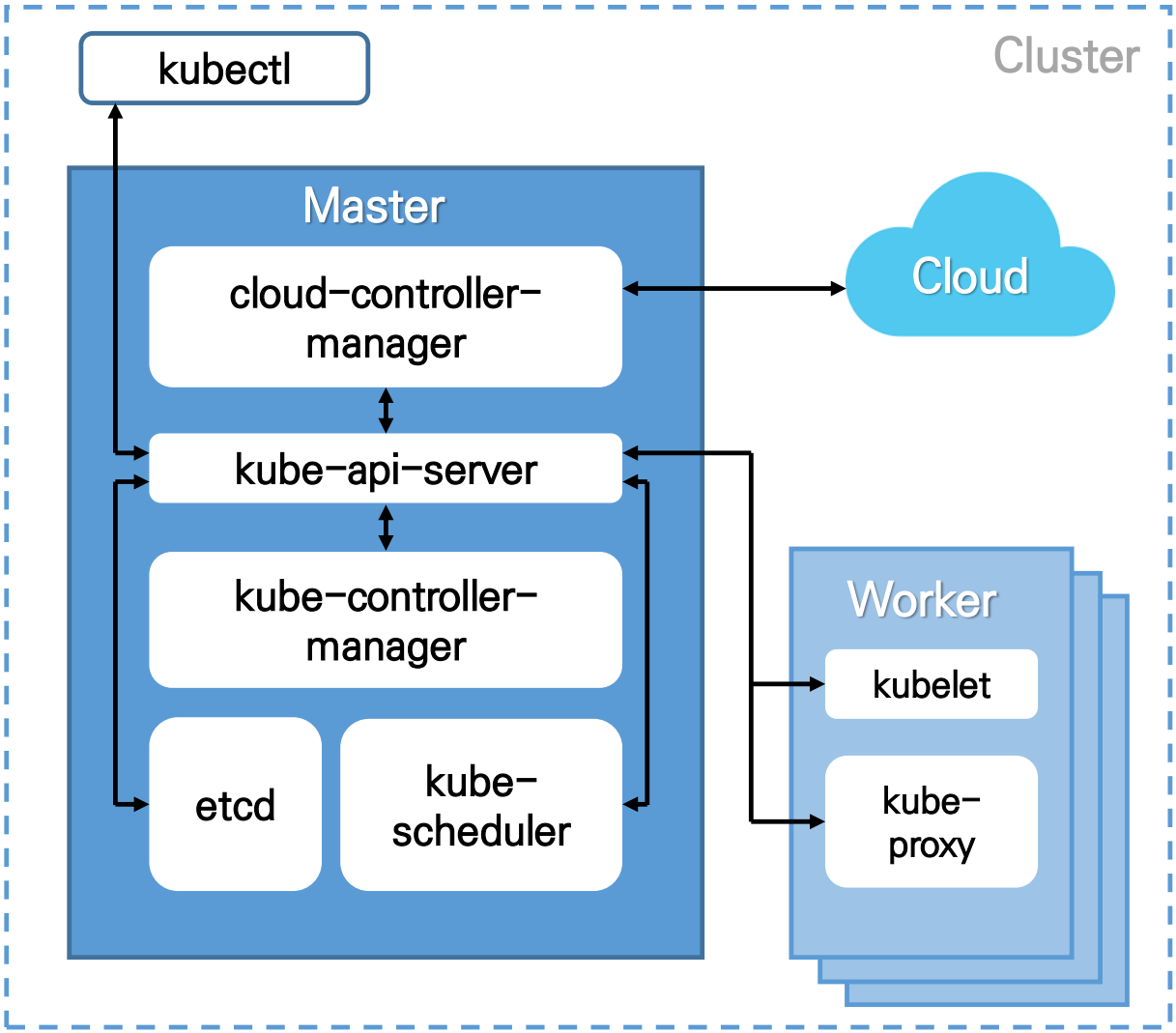
» Kubernetes Cluster 구성



- **kube-api-server**
  쿠버네티스의 모든 통신은 kube-api-server를 통해 통신
- **kube-controller-manager**
  파드들을 관리하는 각각의 컨트롤러를 제어하는 역할
- **kube-scheduler**
  리소스를 자원 할당이 가능한 노드에 할당하는 역할
- **etcd**
  클러스터 내의 모든 세부적인 데이터를 저장하는 키-값 저장소

Kubernetes Cluster 구성 소개

» Kubernetes Cluster 구성



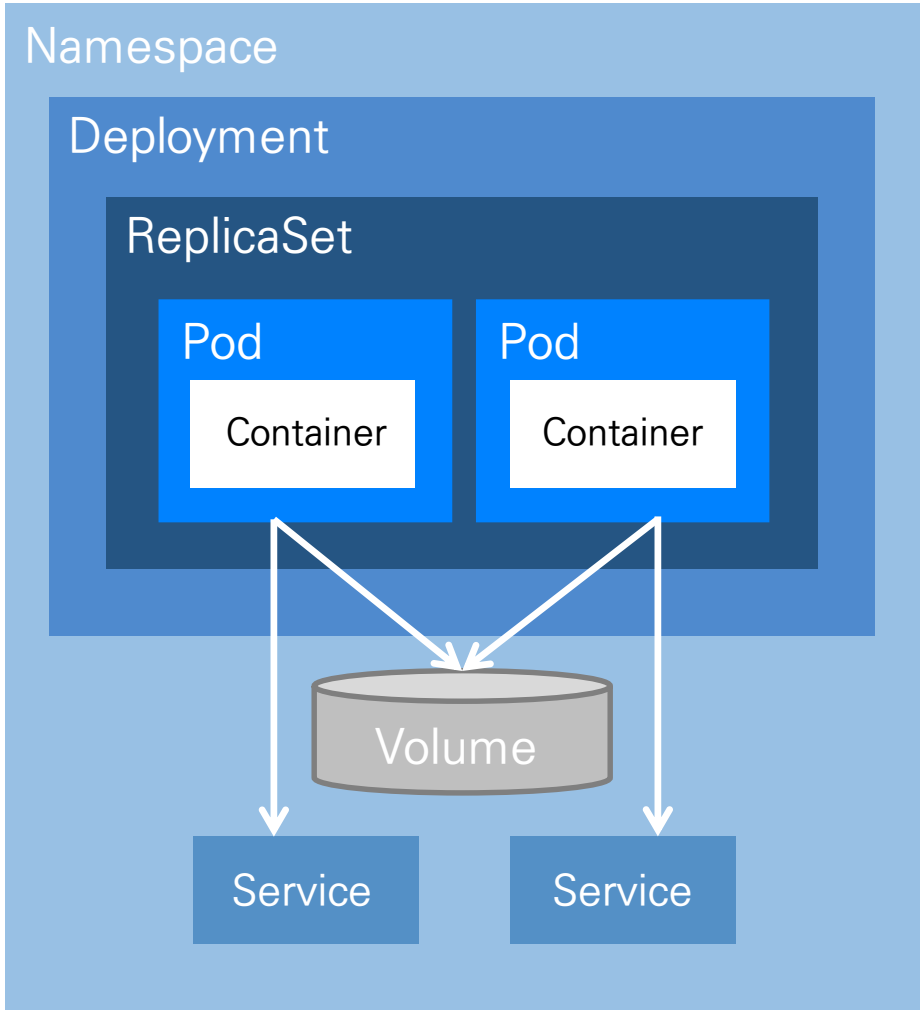
- **cloud-controll-manager**
  컨트롤러들을 클라우드 서비스와 연결하여 관리

- **kubelet**
  모든 노드에서 실행되며 컨테이너 실행 및 지속적인 헬스체크를 통해 마스터의 kube-api-server와 통신을 하는 에이전트

- **kube-proxy**
  클러스터 내부의 가상 네트워크를 설정하여 관리

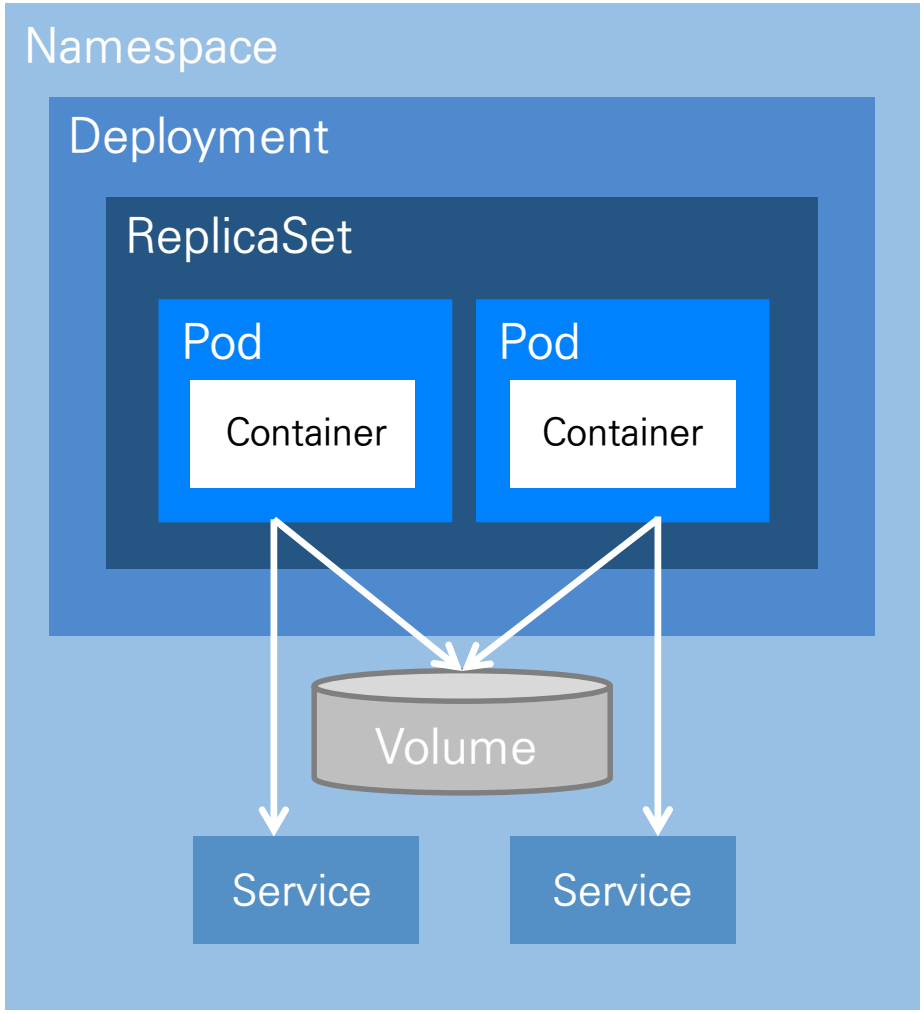
Kubernetes Resource 소개

» Resource 소개



- **Namespace**
  동일한 물리 클러스터 하나를 여러 개의 논리적인 단위로 나누는 리소스
- **Deployment**
  레플리카셋을 관리하면서 실행시켜야 할 파드의 배포 및 관리를 하는 리소스
- **ReplicaSet**
  파드의 개수를 유지하고 관리하는 리소스
- **Pod**
  실제로 컨테이너가 파드에서 실행되며 컨테이너를 돌리는 최소한의 단위

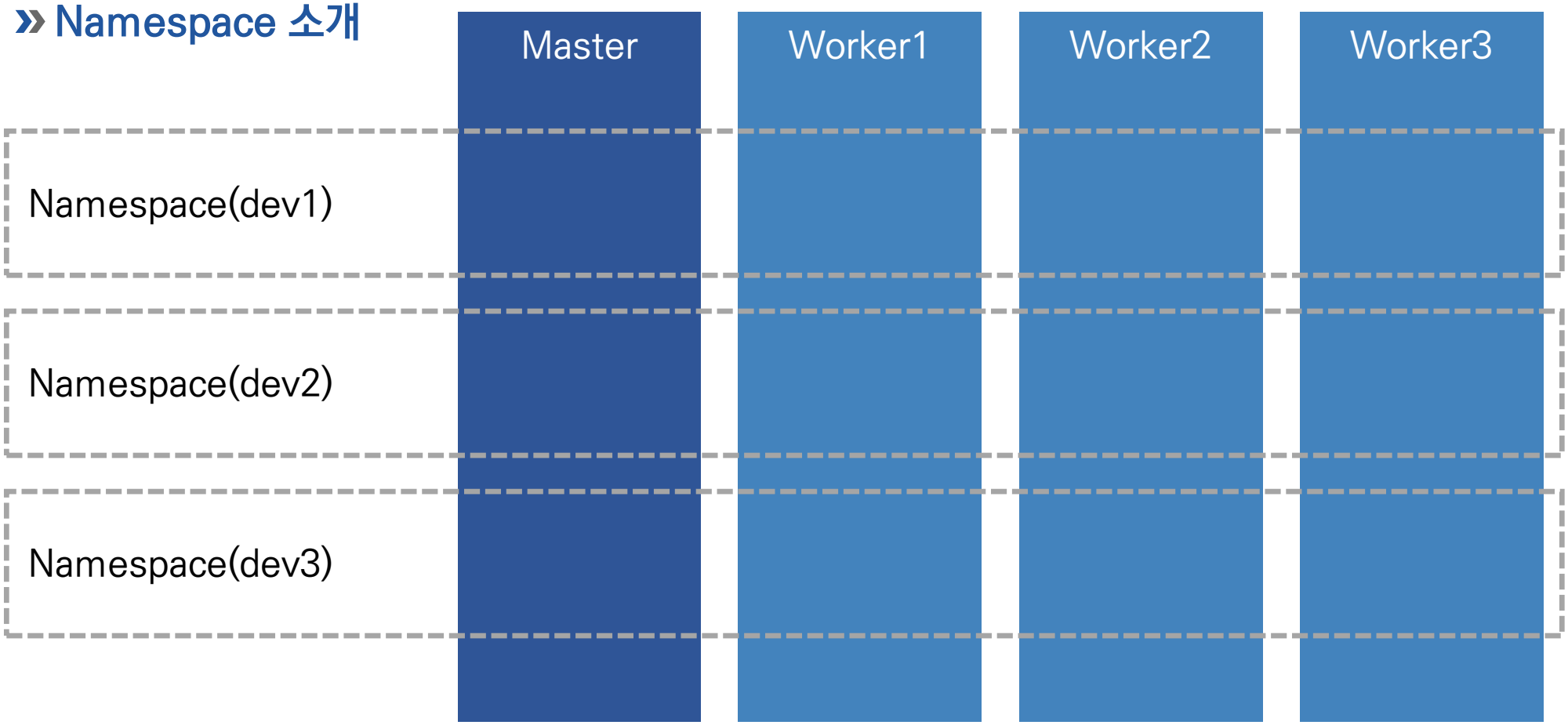
» Resource 소개



- **Service**
  클러스터 외부에서 클러스터 내부 파드에 접근 할 수 있도록 IP를 제공하는 리소스

- **Volume**
  파드의 일부분으로 동일한 파드 내의 컨테이너끼리는 볼륨 공유가 가능하며 컨테이너의 데이터를 보관하여 유지시키는 역할

» Namespace 소개



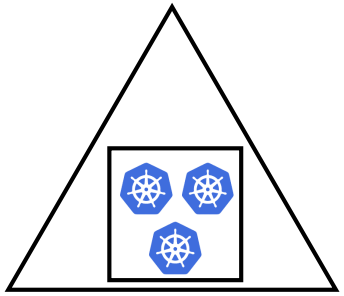
- 네임스페이스는 물리적인 마스터/워커 노드를 논리적인 단위로 분리한 리소스이며 각 사용자는 권한이 부여된 네임스페이스만 접근 가능
  예) dev1 네임스페이스에 대한 권한을 받은 사용자는 dev1만 접근 가능

» Resource 소개

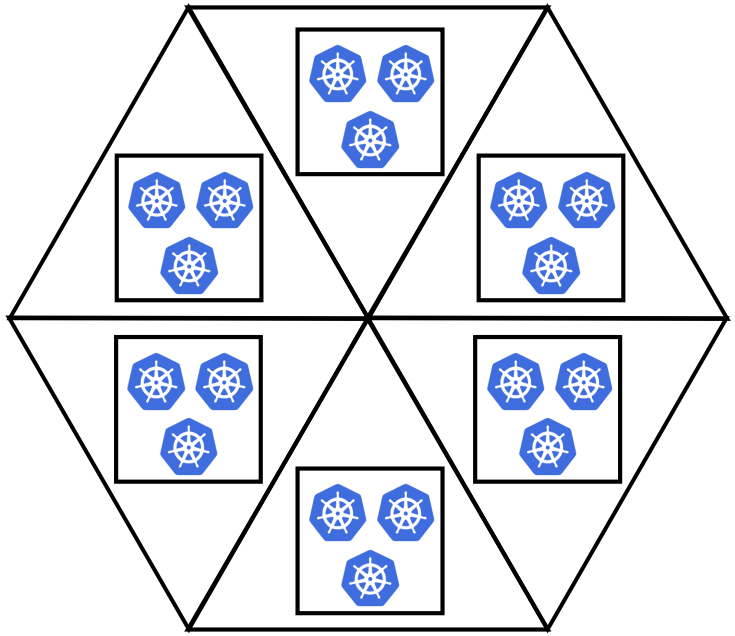
리소스 소개

Container

Pod



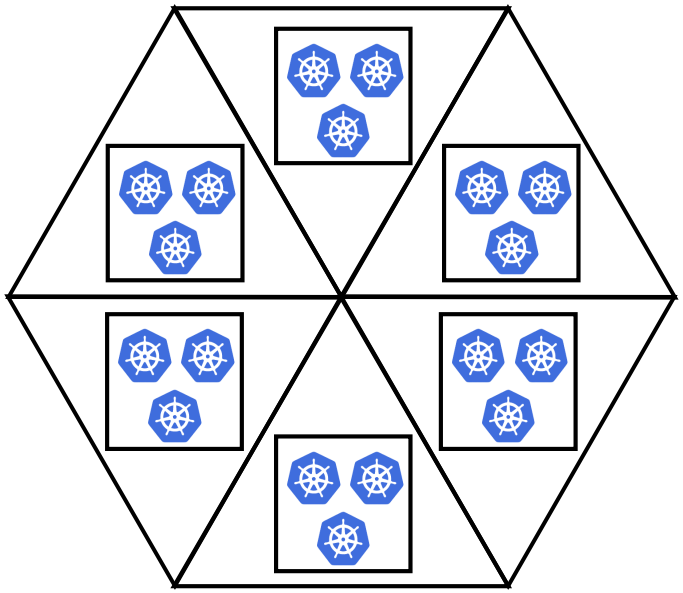
ReplicaSet



Deployment

» Deployment 소개

디플로이먼트 특징

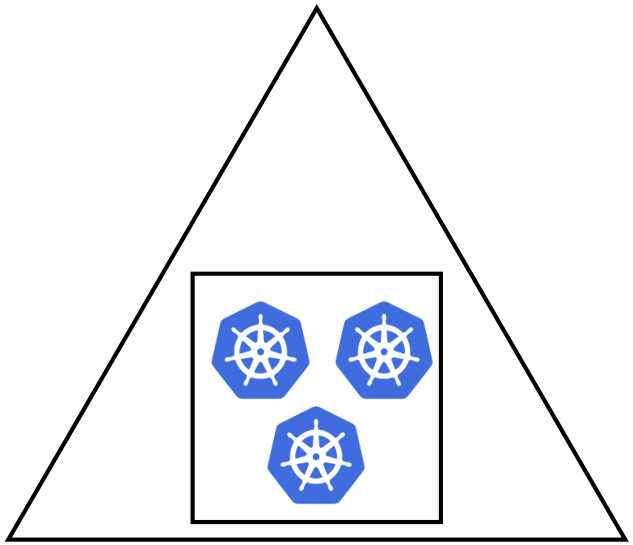


Deployment

- 상태가 없는 앱을 배포할 때 사용되는 가장 기본적인 컨트롤러
- 레플리카셋을 관리
- 앱의 배포를 세밀하게 관리
- 파드 개수 유지
- 배포 시 롤링 업데이트

» ReplicaSet 소개

레플리카셋 특징

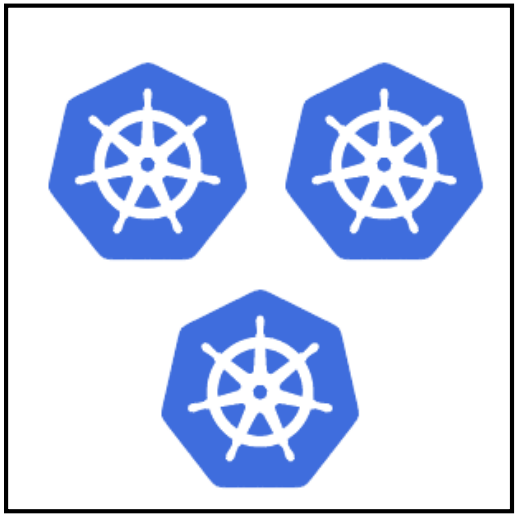


ReplicaSet

- 파드의 개수 유지 및 관리
- 실행되는 파드를 안정적으로 유지
- 파드 개수에 대한 가용성 보장

» Pod 소개

파드 특징

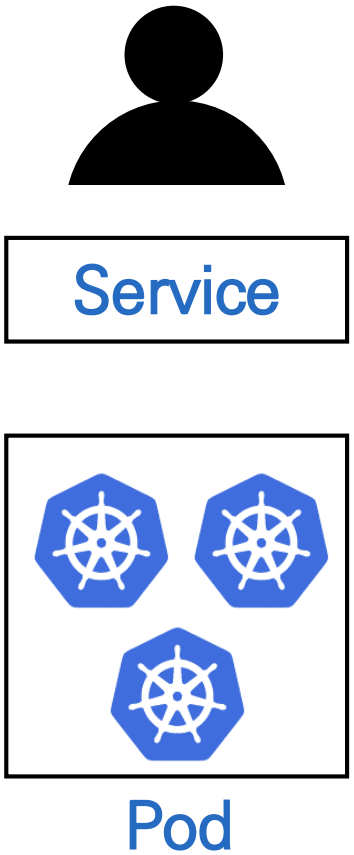


Pod

- 컨테이너가 직접 실행되는 장소
- 컨테이너를 돌리는 최소한의 단위
- 하나 이상의 컨테이너 그룹
- 컨테이너를 직접 관리하지 않고 파드 단위로 관리

» Service 소개

서비스 특징



- 클러스터 외부에서 클러스터 내부 파드에 접근
- 고정된 IP주소 할당
- 파드 간의 로드 밸런싱 지원
- 하나 또는 여러 개의 포트 지원

» Persistent Volume 소개

퍼시스턴트 볼륨 특징

Persistent Volume

- 특정 파드와 상관없이 별도의 생명주기를 가지는 독립적인 볼륨
- 프로비저닝은 정적 방법, 동적 방법 두 가지 방법으로 생성
- PVC로부터 요청이 들어오면 요청에 맞는 PV 할당
- PV와 PVC 매핑은 1:1 관계로 바인딩

» Persistent Volume Claim 소개

퍼시스턴트 볼륨 클레임 특징

Persistent Volume Claim

- 사용자가 PV에 하는 요청
- 파드와 PV를 연결
- 요청 시 스토리지 용량, 접근방법, 셀럭터 등 설정 가능
- PV와 PVC 매핑은 1:1 관계로 바인딩

» StorageClass 소개

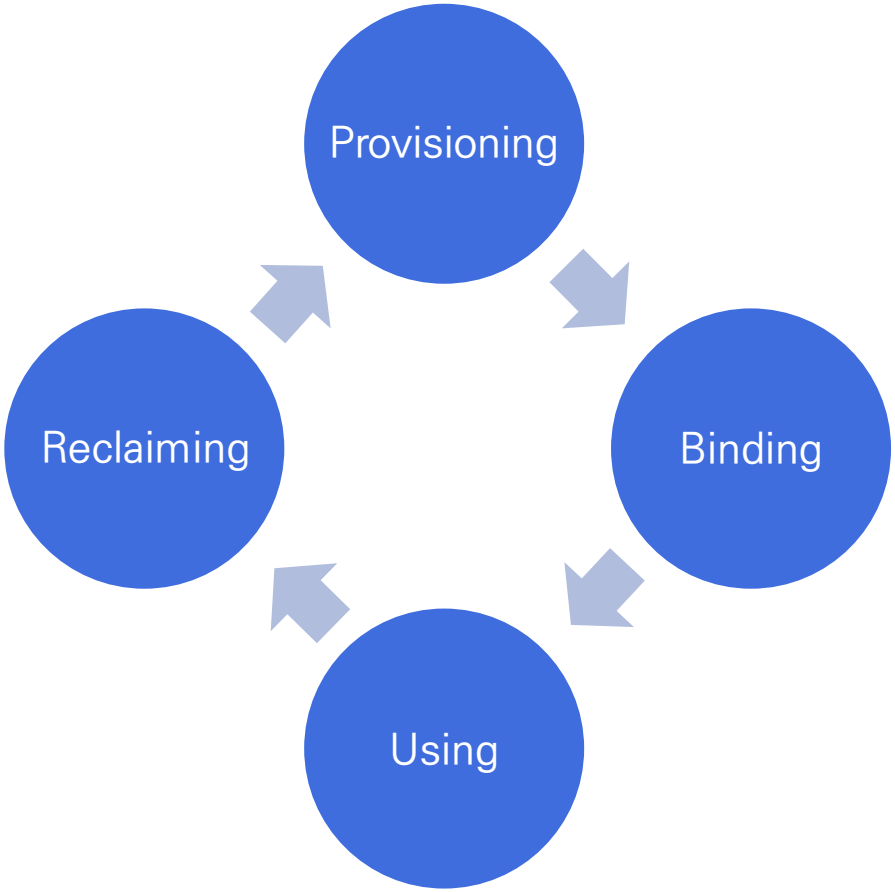
스토리지 클래스 특징

StorageClass

- PV로 확보한 스토리지 종류를 정의 하는 리소스
- PV의 동적 프로비저닝일 때 사용되며 동적 프로비저닝을 활성화 하려면 하나 이상의 스토리지 클래스가 미리 생성 되어 있어야함
- 스토리지 클래스를 미리 정의하면 그에 맞는 PV를 바로 생성할 수 있음

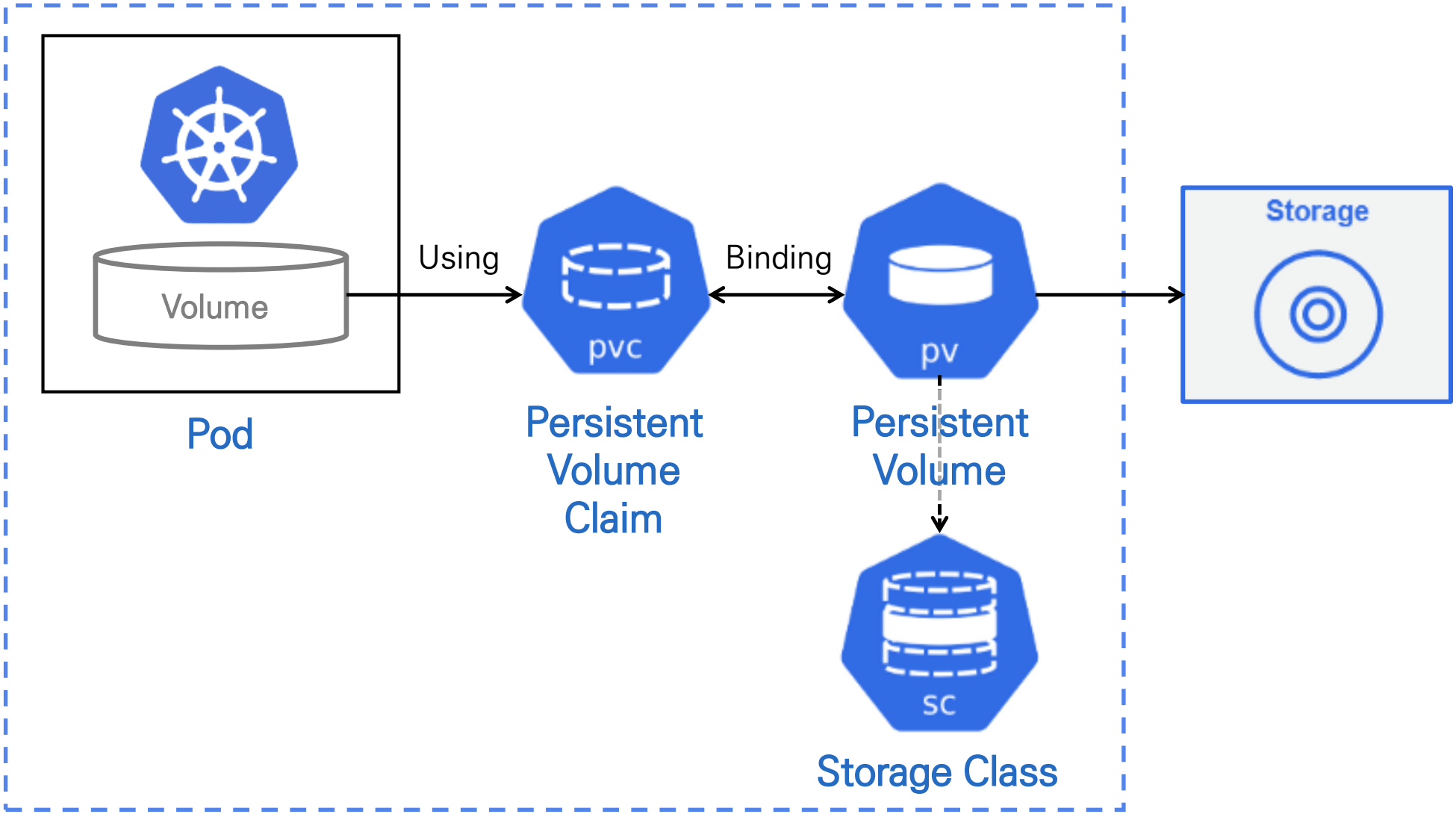
» PV, PVC 생명주기 소개

생명주기



- **Provisioning**
  정적 방법과 동적 방법 두 가지가 있으며 PV를 생성하는 단계
- **Binding**
  PVC가 원하는 접근방법, 스토리지 용량 등을 요청하면 그에 맞는 PV가 연결되는 단계
- **Using**
  파드가 유지되는 동안 파드에서 설정된 PVC가 계속해서 사용되고 있는 단계
- **Reclaiming**
  사용이 끝난 PVC는 삭제되고 사용 중이던 PV는 초기화 되는 과정

» Volume 구조


Volume
Using
pvc
Binding
pv
Storage
Pod
Persistent Volume Claim
Persistent Volume
sc
Storage Class

» ConfigMap 소개

컨피그맵 특징

ConfigMap

- 컨테이너와 분리해서 환경설정 제공
- 키-값 형태의 설정 정보를 저장하는 일종의 저장소 역할
- 간단한 문자부터 큰 설정 파일까지 값으로 가질 수 있음
- 동일한 파드라도 여러 개의 컨피그맵을 사용할 수 있음

» Secret 소개

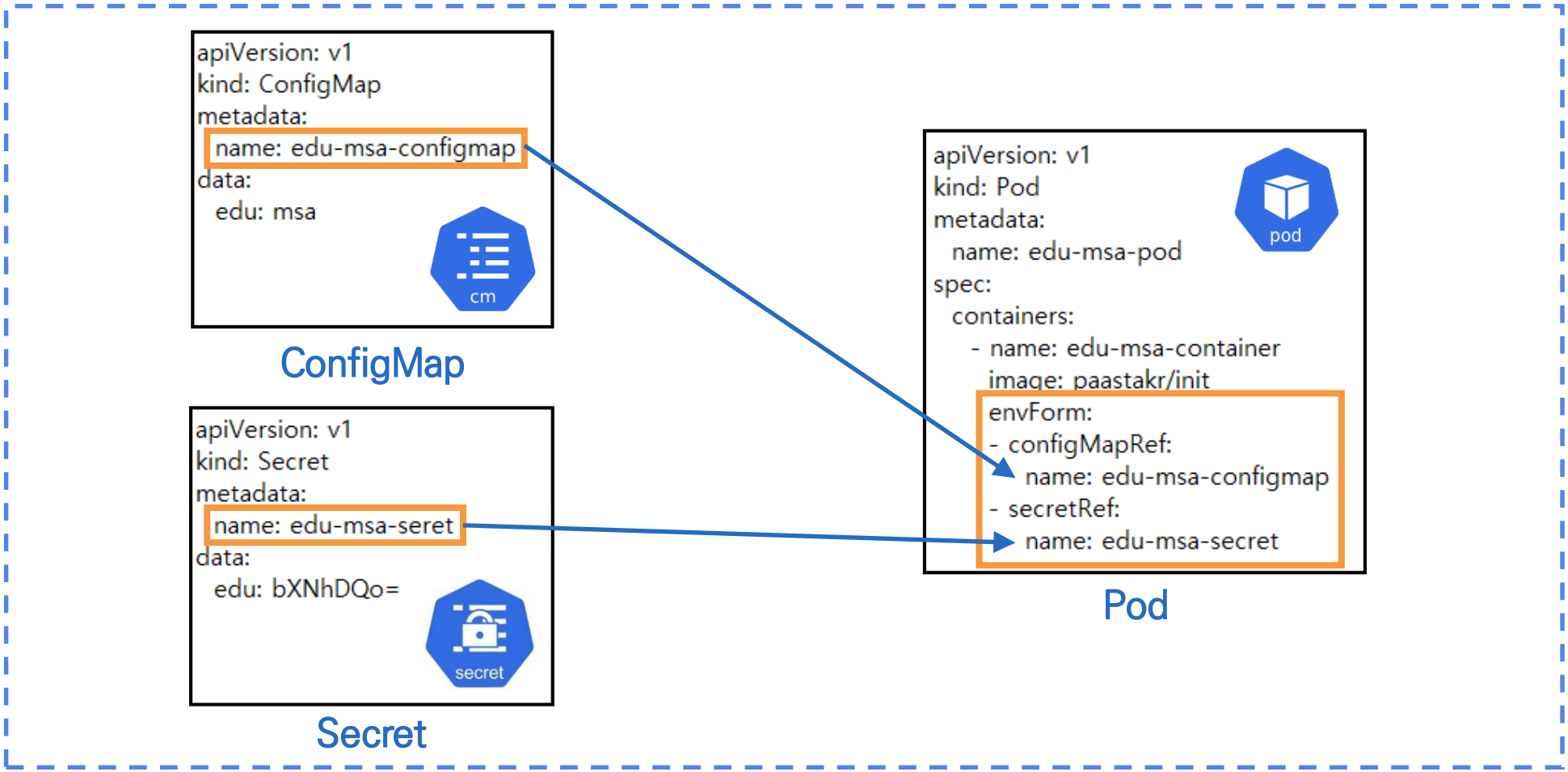
시크릿 특징

Secret

- 보안유지가 필요한 자격증명 및 개인 암호화 키 같은 정보를 저장 및 관리하는 역할
- Base64 인코딩으로 생성
- 키-값 형태의 설정 정보를 저장하는 일종의 저장소 역할
- 메모리에 저장됨
- 개별 시크릿의 최대 크기는 1MB까지 정의
- 모든 파드에는 자동으로 연결된 시크릿 볼륨이 존재

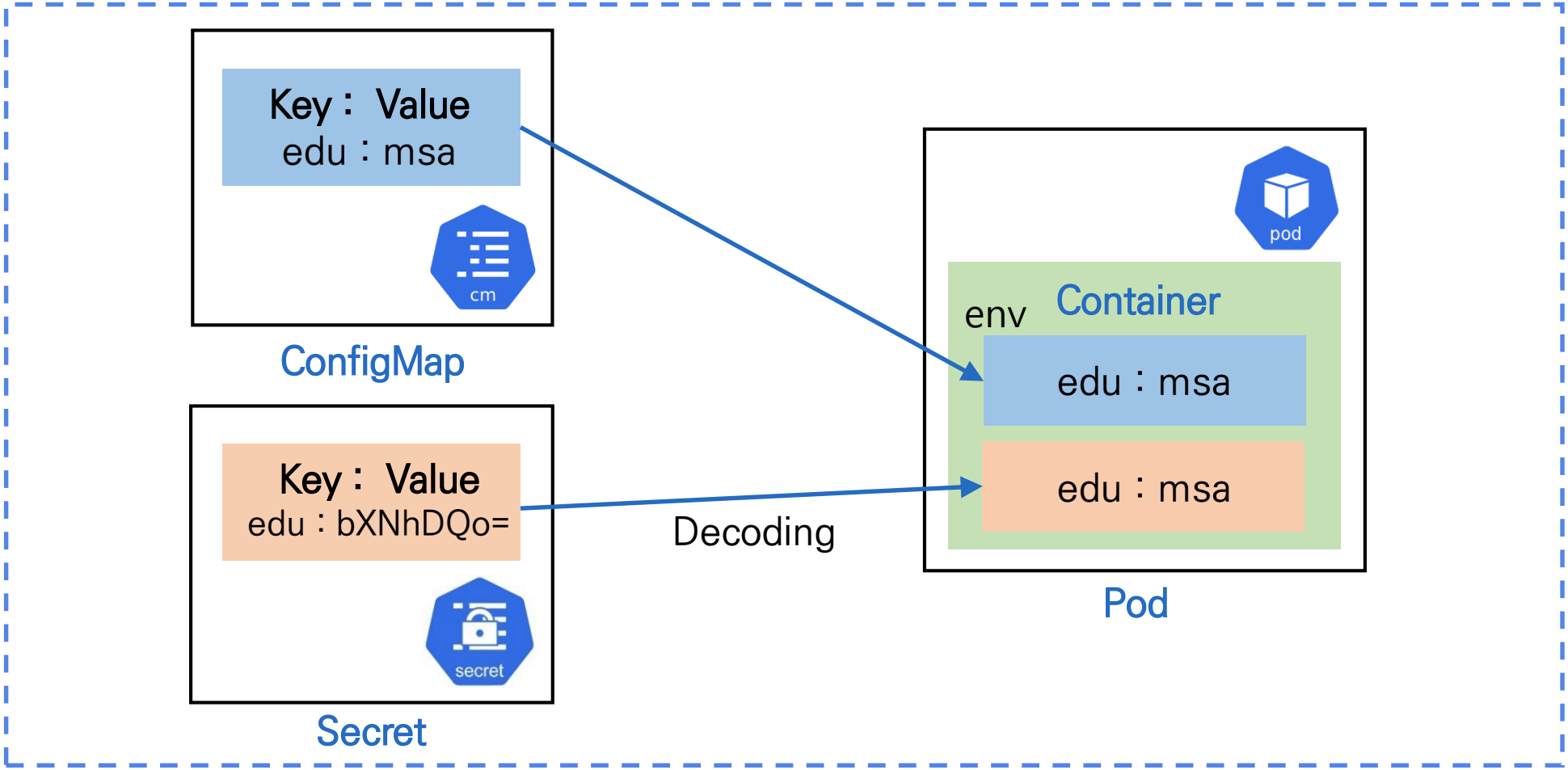
» ConfigMap, Secret 사용 방법 소개

- 파드 생성시 컨피그맵과 시크릿 정보를 입력



» ConfigMap, Secret 사용 방법 소개

- 파드의 컨테이너 환경 변수에 컨피그맵의 데이터와 디코딩 된 시크릿의 데이터가 들어 감



» Kubernetes Resource 관리

쿠버네티스 리소스 관리

- 쿠버네티스 리소스는 YAML 또는 JSON으로 작성된 구성 파일과 함께 kubectl 커맨드 라인 툴을 이용하여 관리

```yaml
apiVersion: v1
kind: Pod
metadata:
 name: paasta-pod
 labels:
   app: paasta-label
spec:
 containers:
 - name: paasta-container
   image: paasta/msa
```

YAML

```json
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "name": "paasta-pod",
    "labels": {
      "app": "paasta-label"
    }
  },
  "spec": {
    "containers": [
      {
        "name": "paasta-container",
        "image": "paasta/msa"
      }
    ]
  }
}
```

JSON

```
I:₩>kubectl create -f paasta.yaml
pod/paasta-pod created

I:₩>kubectl get pods
NAME          READY   STATUS        RESTARTS   AGE
paasta-pod    1/1     Running       0          7s

I:₩>kubectl delete -f paasta.yaml
pod "paasta-pod" deleted
```

kubectl 커맨드

» yaml 파일 작성 방법

디플로이먼트 예시 (1/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- **apiVersion**
  특정 리소스를 생성하기 위해 사용할 쿠버네티스 API 버전을 명세

- **kind**
  어떤 종류의 리소스를 생성하고자 하는지 생성할 리소스 타입을 명세

» yaml 파일 작성 방법

디플로이먼트 예시 (2/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- metadata
  리소스에 이름을 부여하고 리소스를 유일하게 구분 짓는 데이터
- metadata.name
  리소스의 이름을 명세
- metadata.labels
  키-값 쌍으로 구성되어 특정 쿠버네티스 리소스만 나열 또는 검색할 때 사용
- metadata.labels.app
  리소스의 레이블을 설정

» yaml 파일 작성 방법

디플로이먼트 예시 (3/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- spec
  생성하고자 하는 리소스에 대한 내용을 구체적으로 정의
- spec.replicas
  띄우고자 하는 파드의 개수를 설정
- spec.selector
  어떤 파드를 컨트롤하고 감시해야 하는지 정의
- spec.selector.matchLabels
  metadata.labels와 동일한 설정으로 맵핑하여 동일한 레이블을 가진 파드의 컨테이너를 카운팅하여 현재 상태를 측정

» yaml 파일 작성 방법

디플로이먼트 예시 (4/7)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- spec.template
  어떤 파드를 실행할지에 대한 정보를 설정
- spec.template.metadata
  metadata.labels와 동일한 설정으로 맵핑하여 동일한 레이블을 가진 파드를 실행

» yaml 파일 작성 방법

디플로이먼트 예시 (5/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
    - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- spec.template.spec
  실행시킬 컨테이너에 대한 설정
- spec.template.spec.containers[]
  실행시킬 컨테이너의 이름, 이미지, 포트 번호 등을 설정
- spec.template.spec.containers[].name
  컨테이너 이름을 지정

» yaml 파일 작성 방법

디플로이먼트 예시 (6/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
    - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- spec.template.spec.containers[].image
  컨테이너로 배포할 이미지를 지정
- spec.template.spec.containers[].imagePullPolicy
  이미지 다운로드 정책을 지정("Always"는 이미지 여부에 상관없이 다운로드)
- spec.template.spec.containers[].ports[]
  컨테이너 포트번호를 지정

» yaml 파일 작성 방법

디플로이먼트 예시 (7/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
    - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- spec.template.spec.imagePullSecret
  이미지 저장소에 접근하기 위한 인증정보
- spec.template.spec.nodeSelector
  파드를 배포할 특정 노드를 지정

» yaml 파일 작성 방법

서비스 예시 (1/2)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
  - nodePort: ${EDU_MSA_BOARD}
    port: 28082
    protocol: TCP
    targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

- spec.ports.[].nodePort
  외부에서 접근 가능한 포트 번호를 설정
  (외부 포트)
- spec.ports.[].port
  컨테이너 포트 번호를 설정(내부 포트)
- spec.ports[].protocol
  프로토콜 방식을 설정
- spec.ports[].targetport
  접근하고자 하는 컨테이너 포트 번호를 설정

» yaml 파일 작성 방법

서비스 예시 (2/2)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
  - nodePort: ${EDU_MSA_BOARD}
    port: 28082
    protocol: TCP
    targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

- spec.type
  외부에 노출하고자 하는 방식을 설정
  (“NodePort”는 포트 번호를 통해 외부에서 접근할 수 있는 방법)

» yaml 파일 작성 방법

yaml 파일 통합

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
    spec:
      containers:
      - name: board-msa
        image: paastakr/edu-msa-board:latest
        imagePullPolicy: Always
        ports:
        - containerPort: 28082
---
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
  - nodePort: 30201
    port: 28082
    protocol: TCP
    targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

Deployment

Service

- 서로 관련된 여러 yaml 파일을 하나의 파일로 작성하기 위해서는 중간에 “---” 로 구분 지어 작성

» kubectl 명령어 사용법

기본 형태

- kubectl 명령어: https://kubernetes.io/ko/docs/reference/kubectl/_print/

기본 Syntax	$ kubectl [COMMAND] [TYPE] {NAME} {FLAGS}
설명	[COMMAND] 수행하려는 동작 (create, apply, get, describe, delete, edit 등) [TYPE] 리소스 타입 (all, pods, deployments, services 등) {NAME} 리소스 이름 {FLAGS} 선택적 플래그 지정 (-f FILENAME, -n NAMESPACE -o wide 등)

» kubectl 명령어 사용법

생성 (1/2)

기본 Syntax	$ kubectl [COMMAND] {FLAGS} [FILE_NAME]
설명	yaml 파일의 리소스들을 생성 (생성 시 yaml 파일이 있는 위치에서 실행)
사용 예시	$ kubectl apply -f nginx-deployment.yaml

```
I:₩kubeyaml>kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
```

» kubectl 명령어 사용법

생성 (2/2)

기본 Syntax	$ kubectl [COMMAND] {FLAGS} [FILE_NAME]
설명	yaml 파일의 리소스들을 생성 (생성 시 yaml 파일이 있는 위치에서 실행)
사용 예시	$ kubectl create -f nginx-deployment.yaml

```
I:₩kubeyaml>kubectl create -f nginx-deployment.yaml
deployment.apps/nginx created
```

» kubectl 명령어 사용법

조회 (1/4)

기본 Syntax	$ kubectl [COMMAND] [TYPE]
설명	특정 리소스들의 기본 정보 출력
사용 예시	$ kubectl get pods

```
I:₩kubeyaml>kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
nginx-559d75d76b-4zhsz  1/1     Running   0          2m2s
nginx-559d75d76b-7b8s2  1/1     Running   0          2m2s
```

» kubectl 명령어 사용법

조회 (2/4)

기본 Syntax	$ kubectl [COMMAND] [TYPE] {FLAGS}
설명	특정 리소스들의 추가 정보 출력
사용 예시	$ kubectl get pods -o wide

```
I:₩kubeyaml>kubectl get pods -o wide
NAME                    READY   STATUS    RESTARTS   AGE     IP              NODE                   NOMINATED NODE   READINESS GATES
nginx-559d75d76b-4zhsz  1/1     Running   0          2m17s   10.233.117.192  playpark-caas-worker-6 <none>           <none>
nginx-559d75d76b-7b8s2  1/1     Running   0          2m17s   10.233.117.191  playpark-caas-worker-6 <none>           <none>
```

03 kubectl 명령어 이해

» kubectl 명령어 사용법

조회 (3/4)

기본 Syntax	$ kubectl [COMMAND] [TYPE] {NAME}
설명	특정 이름을 가진 리소스들의 기본 정보 출력
사용 예시	$ kubectl get pod nginx-559d75d76b-4zhsz

```
I:₩kubeyaml>kubectl get pod nginx-559d75d76b-4zhsz
NAME                    READY   STATUS    RESTARTS   AGE
nginx-559d75d76b-4zhsz  1/1     Running   0          5m14s
```

03 kubectl 명령어 이해

» kubectl 명령어 사용법

조회 (4/4)

기본 Syntax	$ kubectl [COMMAND] [TYPE] {NAME} {FLAGS}
설명	특정 이름을 가진 리소스들의 추가 정보 출력
사용 예시	$ kubectl get pod nginx-559d75d76b-4zhsz -o wide

```
I:₩kubeyaml>kubectl get pod nginx-559d75d76b-4zhsz -o wide
NAME                    READY   STATUS    RESTARTS   AGE     IP              NODE                    NOMINATED NODE   READINESS GATES
nginx-559d75d76b-4zhsz  1/1     Running   0          5m20s   10.233.117.192  playpark-caas-worker-6  <none>           <none>
```

» kubectl 명령어 사용법

상세 조회

기본 Syntax	$ kubectl [COMMAND] [TYPE] {NAME}
설명	특정 이름을 가진 리소스의 자세한 정보 출력
사용 예시	$ kubectl describe pod nginx-559d75d76b-4zhsz

```
I:₩kubeyaml>kubectl describe pod nginx-559d75d76b-4zhsz
Name:         nginx-559d75d76b-4zhsz
Namespace:    paas-83cc724a-4425-4f1e-aab4-63a4f23fcd99-caas
Priority:     0
Node:         playpark-caas-worker-6/10.0.50.254
Start Time:   Fri, 29 Oct 2021 14:47:49 +0900
Labels:       app=nginx
              pod-template-hash=559d75d76b
Annotations:  cni.projectcalico.org/podIP: 10.233.117.192/32
              cni.projectcalico.org/podIPs: 10.233.117.192/32
              kubernetes.io/limit-ranger: LimitRanger plugin set: cpu, memory request for container nginx; cpu, memory limit for container nginx
Status:       Running
IP:           10.233.117.192
IPs:
  IP:           10.233.117.192
Controlled By:  ReplicaSet/nginx-559d75d76b
Containers:
```

» kubectl 명령어 사용법

로그 조회

기본 Syntax	$ kubectl [COMMAND] {FLAGS} {POD_NAME}
설명	특정 이름을 가진 리소스의 로그 정보 출력
사용 예시	$ kubectl logs -f edu-msa-zuul-647b787d7-g5st7

```
I:₩kubeyaml>kubectl logs -f edu-msa-zuul-647b787d7-g5st7

  .   ____          _            __ _ _
 /₩₩ / ___'_ __ _ _(_)_ __  __ _ ₩ ₩ ₩ ₩
( ( )₩___ | '_ | '_| | '_ ₩/ _` | ₩ ₩ ₩ ₩
 ₩₩/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_₩__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::  (v2.3.8.BUILD-SNAPSHOT)

2021-10-29 09:15:55.805  INFO 1 --- [           main] paasta.msa.zuul.ZuulApplication          : No active profile set,
falling back to default profiles: default
2021-10-29 09:17:06.104  WARN 1 --- [           main] o.s.boot.actuate.endpoint.EndpointId     : Endpoint ID 'hystrix.st
ream' contains invalid characters, please migrate to a valid format.
```

» kubectl 명령어 사용법

Editor 수정 (1/2)

기본 Syntax	$ kubectl [COMMAND] [TYPE] {NAME}
설명	특정 이름을 가진 리소스 정보 수정
사용 예시	$ kubectl edit deployment nginx

- 위 명령어 입력 시 수정하려는 리소스의 정보가 적힌 Editor가 열림
- 수정 후 Editor 저장 및 종료 시 수정 내용 반영

» kubectl 명령어 사용법

Editor 수정 (2/2)

```
spec:
  progressDeadlineSeconds: 600
  replicas: 2
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: nginx
  strategy:
```

```
spec:
  progressDeadlineSeconds: 600
  replicas: 3
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: nginx
  strategy:
```

```
I:₩kubeyaml>kubectl edit deployment nginx
deployment.apps/nginx edited

I:₩kubeyaml>kubectl get replicasets
NAME                DESIRED   CURRENT   READY   AGE
nginx-559d75d76b    3         3         3       47m
```

» kubectl 명령어 사용법

yaml 파일 수정 (1/2)

- 생성시 배포한 yaml 파일을 수정하여 다시 재배포

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.13.12
        ports:
        - containerPort: 80
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 4
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.13.12
        ports:
        - containerPort: 80
```

» kubectl 명령어 사용법

yaml 파일 수정 (2/2)

기본 Syntax	$ kubectl [COMMAND] {FLAGS} {NAME}
설명	yaml 파일을 이용한 리소스 수정 (수정 시 yaml 파일이 있는 위치에서 실행)
사용 예시	$ kubectl apply -f nginx-deployment.yaml

```
I:₩kubeyaml>kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx configured

I:₩kubeyaml>kubectl get replicasets
NAME                  DESIRED   CURRENT   READY   AGE
nginx-559d75d76b      4         4         4       30m
```

» kubectl 명령어 사용법

삭제 (1/2)	
기본 Syntax	$ kubectl [COMMAND] [TYPE] {NAME}
설명	삭제하고자 하는 특정 리소스 삭제
사용 예시	$ kubectl delete pod nginx-559d75d76b-4zhsz

```
I:₩kubeyaml>kubectl delete pod nginx-559d75d76b-4zhsz
pod "nginx-559d75d76b-4zhsz" deleted
```

» kubectl 명령어 사용법

삭제 (2/2)

기본 Syntax	$ kubectl [COMMAND] {FLAGS} [FILE_NAME]
설명	yaml 파일을 이용한 리소스 삭제 (삭제 시 yaml 파일이 있는 위치에서 실행)
사용 예시	$ kubectl delete -f nginx-deployment.yaml

```
I:₩kubeyaml>kubectl delete -f nginx-deployment.yaml
deployment.apps "nginx" deleted
```

kubectl 명령어 이해

Context 관련 명령어

- kubectl config current-context 현재 컨택스트 확인
- kubectl config get-contexts 컨택스트 목록 조회
- kubectl config use-context docker-desktop 컨택스트 변경
- kubectl config view 컨택스트 상세 보기

Kubeconfig 파일 위치

- ~/.kube/config

kubectl 명령어 이해

Namespace 관련 명령어 (kafka 네임스페이스 가정)

- kubectl get ns 네임스페이스 조회
- kubectl create namespace kafka 네임스페이스 생성
- kubectl delete namespace kafka 네임스페이스 삭제
- kubectl get pods -n kafka 네임스페이스내 pod 조회
- kubectl config set-context --current --namespace=kafka 기본 네임스페이스 변경
- kubectl apply -f deployment.yaml -n kafka 네임스페이스에 리소스 생성
- kubectl describe namespace kafka 네임스페이스 상세정보 확인
- kubectl get all --all-namespaces 네임스페이스별 리소스 확인
- kubectl delete all --all -n kafka 네임스페이스내 리소스 삭제

감사합니다

THANK YOU